



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

**Кафедра Математического обеспечения и стандартизации информационных
технологий**

Отчет по практической работе №1

по дисциплине

«Технология разработки программных приложений»

Тема: «Системы контроля версий»

Выполнил:

Студент группы ИНБО-20-23

Боргачев Тимофей Максимович

Проверил:

ассистент Петрова А.А.

Москва 2025

ОГЛАВЛЕНИЕ

ПРАКТИЧЕСКАЯ РАБОТА №1. СИСТЕМЫ КОНТРОЛЯ ВЕРСИЙ.....	3
Введение	3
Часть 1. Основные команды Git	3
Часть 2. Системы управления репозиториями.....	15
Часть 3. Работа с ветвлением и оформление кода.....	22
Ответы на вопросы.....	30
Вывод	30

ПРАКТИЧЕСКАЯ РАБОТА №1. СИСТЕМЫ КОНТРОЛЯ ВЕРСИЙ

Введение

Цель работы: получить навыки по работе с командной строкой и git'ом.

В рамках работы рассматриваются базовые команды для управления локальным репозиторием, работа с ветвлением, слиянием изменений и откатом к предыдущим состояниям проекта.

Часть 1. Основные команды Git

Первым шагом при выполнении работы является установка клиента git. В случае с операционной системой Windows – это Git Bash.

После того, как все действия по установке завершены, убедимся, что Git появился в системе компьютера. Для этого в терминал необходимо ввести команду `git --version` и, в случае корректной установки, получить ответ в виде текущей установленной версии. Результат выполнения команды представлен на рис. 1.



```
MINGW64:/c/Users/borga
borga@DESKTOP-BB86VVG MINGW64 ~
$ git --version
git version 2.48.1.windows.1
borga@DESKTOP-BB86VVG MINGW64 ~
$
```

Рисунок 1 – Команда `git --version`

Следующий шаг – настройка Git. Необходимо выполнить следующие команды:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@whatever.com"
```

где `Your Name` – ваш логин, `your_email@whatever.com` – ваша почта

Затем выполнить следующие команды:

```
git config --global core.autocrlf true
git config --global core.safecrlf warn
```

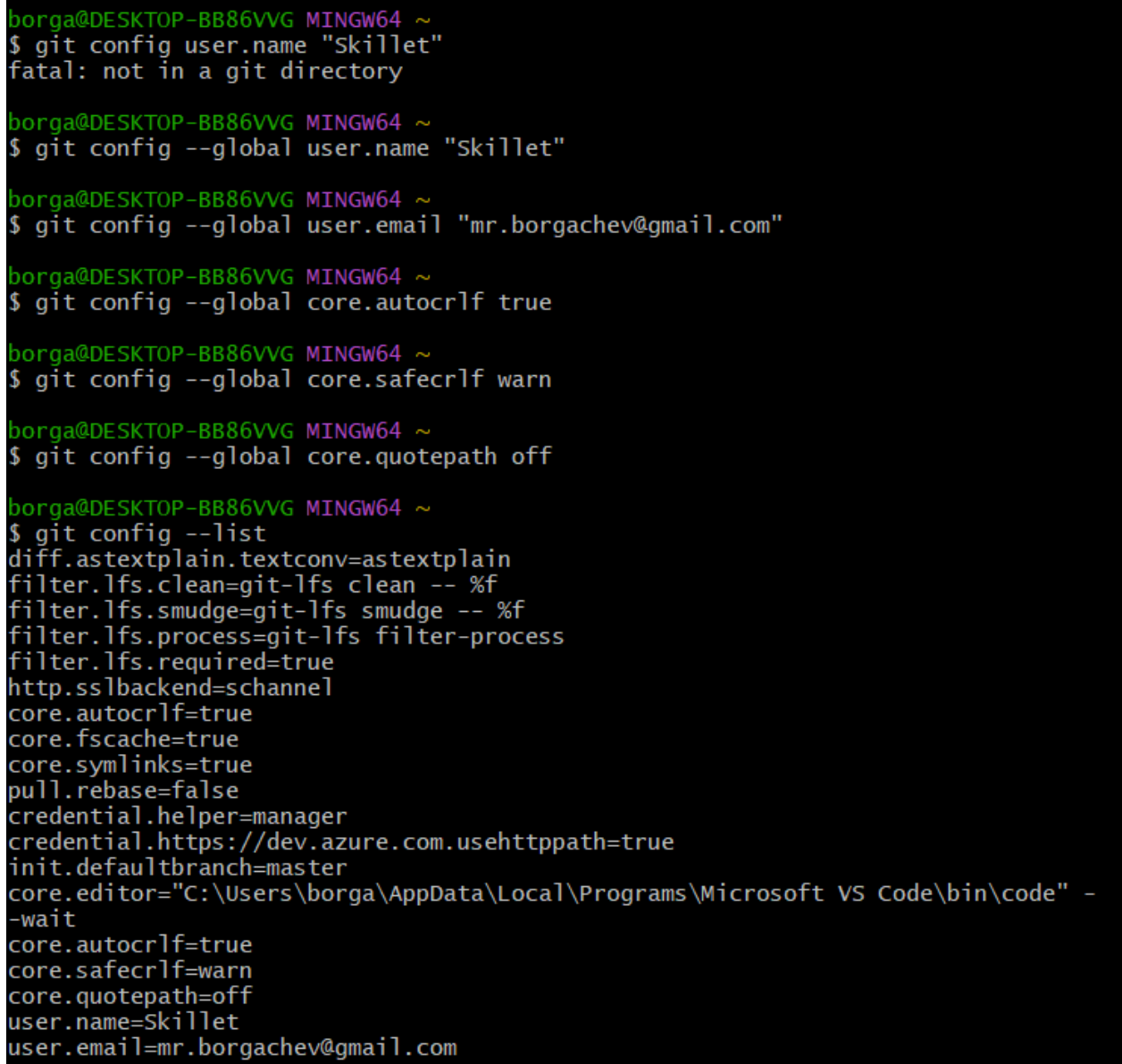
А также ввести команду:

```
git config --global core.quotepath off
```

Для проверки верности всех введенных команд, нужно ввести:

```
git config -list
```

Корректный результат выполнения всех команд представлен на рис. 2.



```
borga@DESKTOP-BB86VVG MINGW64 ~
$ git config user.name "Skillet"
fatal: not in a git directory

borga@DESKTOP-BB86VVG MINGW64 ~
$ git config --global user.name "Skillet"

borga@DESKTOP-BB86VVG MINGW64 ~
$ git config --global user.email "mr.borgachev@gmail.com"

borga@DESKTOP-BB86VVG MINGW64 ~
$ git config --global core.autocrlf true

borga@DESKTOP-BB86VVG MINGW64 ~
$ git config --global core.safecrlf warn

borga@DESKTOP-BB86VVG MINGW64 ~
$ git config --global core.quotepath off

borga@DESKTOP-BB86VVG MINGW64 ~
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=true
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
core.editor="C:\Users\borga\AppData\Local\Programs\Microsoft VS Code\bin\code" -
-wait
core.autocrlf=true
core.safecrlf=warn
core.quotepath=off
user.name=Skillet
user.email=mr.borgachev@gmail.com
```

Рисунок 2 – Настройка Git

Третий шаг – начало работы, а именно создание папки и файлов. Для создания локального репозитория необходимо перейти в желаемую директорию с помощью команды *cd <путь к директории>* и создать папку с помощью команды *mkdir <Название папки>*. Результат создания

представлен на рис. 3.

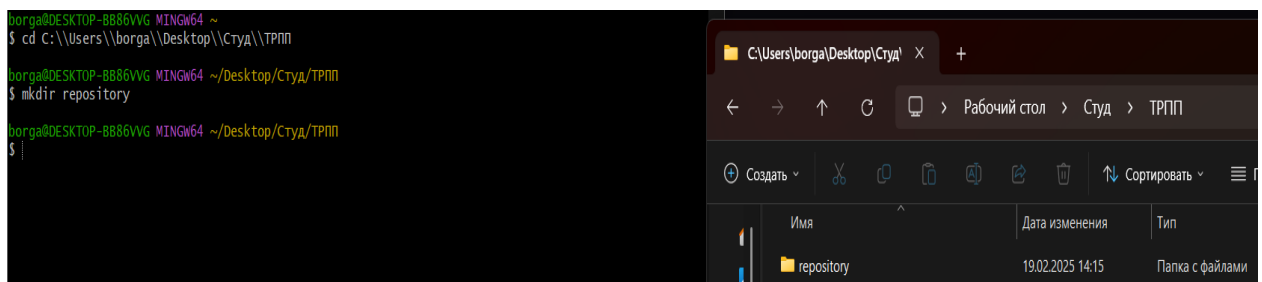


Рисунок 3 – Создание папки с помощью Git Bash

Создадим файлы `File.html`, `File.docx` и `File.pdf` в папке `repository`. Это можно сделать с помощью команды ***touch*** *<имя файла>*. Результат создания файлов представлен на рис 4.

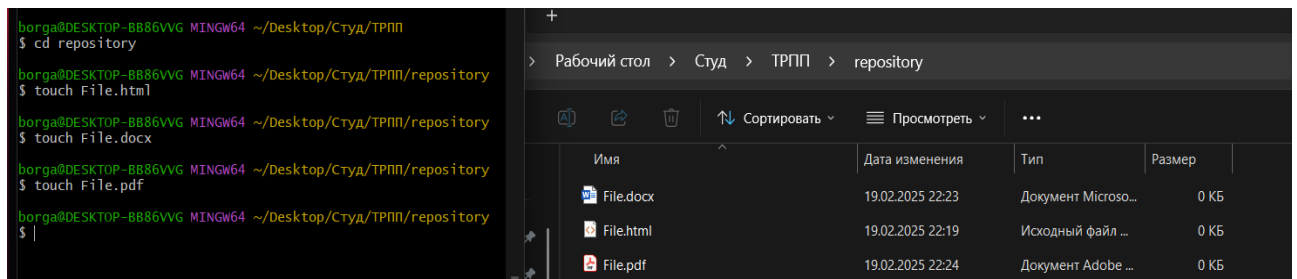


Рисунок 4 – Создание файлов с помощью Git Bash

Для внесения изменений в файл, можно воспользоваться командой ***nano***. Процесс внесения изменений в файл `File.html` представлен на рис. 5.

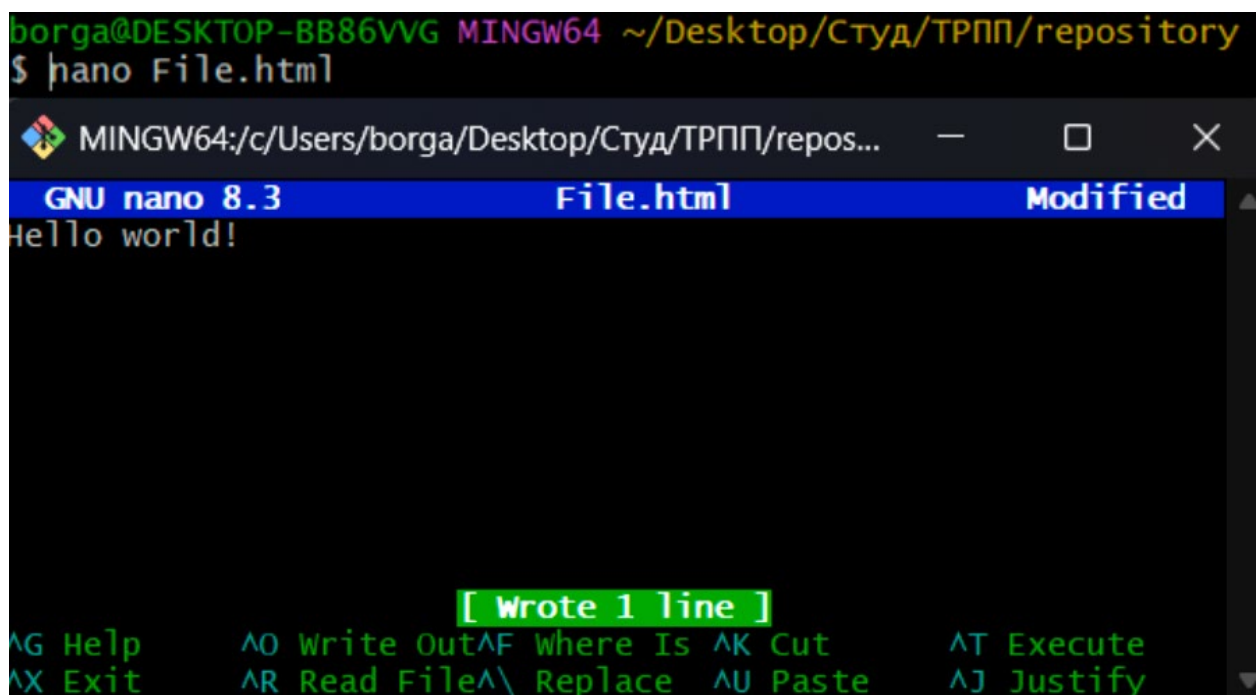


Рисунок 5 – Процесс внесения изменений в файл с помощью Git Bash

Чтобы создать репозиторий в текущей папке, необходимо использовать команду *git init*. Результат выполнения команды представлен на рис. 6.

```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/repository
$ git init
Initialized empty Git repository in C:/Users/borga/Desktop/Студ/ТРПП/repository/.git/

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/repository (master)
$
```

Рисунок 6 – Создание локального репозитория

На следующем тапе работы нужно сделать коммит в репозиторий ранее измененного файла. Для этого нужно использовать команды:

```
git add <Название вашего файла>
git commit -m "Ваш текст для коммита"
```

Результат выполнения коммита представлен на рис. 7.

```
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git add File.html
warning: in the working copy of 'File.html', LF will be replaced by CRLF the next time Git touches it

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git commit -m "Hello world!"
[master (root-commit) bba69ea] Hello world!
1 file changed, 1 insertion(+)
create mode 100644 File.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$
```

Рисунок 7 – Коммит файла в репозиторий

Проверка состояния репозитория представлена на рис. 8.

```
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    File.docx
    File.pdf

nothing added to commit but untracked files present (use "git add" to track)

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$
```

Рисунок 8 – Проверка статуса репозитория

Проверка состояния репозитория после добавления изменений в файл, представлена на рис. 9.

```

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ nano File.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   File.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        File.docx
        File.pdf

no changes added to commit (use "git add" and/or "git commit -a")

```

Рисунок 9 – Проверка статуса репозитория после изменения файла

Для индексации изменений необходимо выполнить команды:

```

git add <Имя вашего файла>
git status

```

Результат представлен на рис. 10.

```

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git add File.html
warning: in the working copy of 'File.html', LF will be replaced by CRLF the next time Git touches it

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git add File.docx

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git add File.pdf

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   File.docx
        modified:   File.html
        new file:   File.pdf

```

Рисунок 10 – Индексация изменений

Создадим в директории еще 3 файла при помощи команды *touch <имя файла>*. Выполнение команд представлено на рис. 11.

```
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ touch a.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ touch b.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ touch v.html
```

Рисунок 11 – Создание файлов в репозитории

В Git присутствует возможность разделения индексации и коммитов. Проиндексируем первые два файла и сделаем их коммит отдельно от третьего. Результат представлен на рис. 12.

```
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git add a.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git add b.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git commit -m "a+b files"
[master 4b5b90e] a+b files
5 files changed, 1 insertion(+), 1 deletion(-)
create mode 100644 File.docx
create mode 100644 File.pdf
create mode 100644 a.html
create mode 100644 b.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git add v.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/repository (master)
$ git commit -m "v file"
[master 7427b42] v file
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 v.html
```

Рисунок 12 – Разделение индексации от коммитов

Проверка состояния репозитория после нескольких изменений файла представлена на рис. 13.


```

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпн/repository (master)
$ nano File.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпн/repository (master)
$ git add File.html
warning: in the working copy of 'File.html', LF will be replaced by CRLF the next time Git touches it

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпн/repository (master)
$ nano File.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпн/repository (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   File.html

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   File.html

```

Рисунок 13 – Проверка статуса репозитория после двух изменений файла

Проверка состояния репозитория после коммита дважды измененного файла представлена на рис. 14.

```

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпн/repository (master)
$ git commit -m "Two changes"
[master a2b4a11] Two changes
1 file changed, 14 insertions(+), 1 deletion(-)

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпн/repository (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   File.html

no changes added to commit (use "git add" and/or "git commit -a")

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпн/repository (master)
$ git add .
warning: in the working copy of 'File.html', LF will be replaced by CRLF the next time Git touches it

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпн/repository (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   File.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпн/repository (master)
$ git commit -m "Second change"
[master e138e16] Second change
1 file changed, 3 insertions(+), 10 deletions(-)

```

Рисунок 14 – Проверка статуса репозитория после коммита дважды измененного файла

Для просмотра истории коммитов используется команда:

```
git log
```

Результат выполнения команды представлен на рис. 15.

```
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трнп/repository (master)
$ git log
commit e138e16bc604bab083bc331be72a29fee5abb22 (HEAD -> master)
Author: Skillet <mr.borgachev@gmail.com>
Date: Thu Feb 20 15:32:40 2025 +0300

    Second change

commit a2b4a11f9076503cbeb30ab8323d26a15a03500c
Author: Skillet <mr.borgachev@gmail.com>
Date: Thu Feb 20 15:31:44 2025 +0300

    Two changes

commit 7427b42da3fcbfba26003a5306d4d38d854b51e8
Author: Skillet <mr.borgachev@gmail.com>
Date: Thu Feb 20 15:21:39 2025 +0300

    v file

commit 4b5b90ecb07636a86e907671482fadd946620060
Author: Skillet <mr.borgachev@gmail.com>
Date: Thu Feb 20 15:21:12 2025 +0300

    a+b files

commit bba69eafe83741d693fafdf6d5b8bc129b4df746
Author: Skillet <mr.borgachev@gmail.com>
Date: Thu Feb 20 14:36:01 2025 +0300

    Hello world!
```

Рисунок 15 – Просмотр истории коммитов

Команда *log* позволяет контролировать формат выводимой информации. Пример различных параметров логирования представлен на рис. 16.

```

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трnn/repository (master)
$ git log --pretty=oneline
e138e16bc604babc083bc331be72a29fee5abb22 (HEAD -> master) Second change
a2b4a11f9076503cbeb30ab8323d26a15a03500c Two changes
7427b42da3fcbfba26003a5306d4d38d854b51e8 v file
4b5b90ecb07636a86e907671482fadd946620060 a+b files
bba69eafe83741d693fafdf6d5b8bc129b4df746 Hello World!
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трnn/repository (master)
$ git log --pretty=oneline --max-count=2
e138e16bc604babc083bc331be72a29fee5abb22 (HEAD -> master) Second change
a2b4a11f9076503cbeb30ab8323d26a15a03500c Two changes
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трnn/repository (master)
$ git log --pretty=oneline --since='5 minutes ago'
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трnn/repository (master)
$ git log --pretty=oneline --until='5 minutes ago'
e138e16bc604babc083bc331be72a29fee5abb22 (HEAD -> master) Second change
a2b4a11f9076503cbeb30ab8323d26a15a03500c Two changes
7427b42da3fcbfba26003a5306d4d38d854b51e8 v file
4b5b90ecb07636a86e907671482fadd946620060 a+b files
bba69eafe83741d693fafdf6d5b8bc129b4df746 Hello World!
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трnn/repository (master)
$ git log --pretty=oneline --author=Skillet
e138e16bc604babc083bc331be72a29fee5abb22 (HEAD -> master) Second change
a2b4a11f9076503cbeb30ab8323d26a15a03500c Two changes
7427b42da3fcbfba26003a5306d4d38d854b51e8 v file
4b5b90ecb07636a86e907671482fadd946620060 a+b files
bba69eafe83741d693fafdf6d5b8bc129b4df746 Hello World!
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трnn/repository (master)
$ git log --pretty=oneline --all
e138e16bc604babc083bc331be72a29fee5abb22 (HEAD -> master) Second change
a2b4a11f9076503cbeb30ab8323d26a15a03500c Two changes
7427b42da3fcbfba26003a5306d4d38d854b51e8 v file
4b5b90ecb07636a86e907671482fadd946620060 a+b files
bba69eafe83741d693fafdf6d5b8bc129b4df746 Hello World!

```

Рисунок 16 – Различные параметры логирования

Для уточнения формата используется флаг *format*. Пример использования логирования с форматом представлен на рис. 17.

```

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трnn/repository (master)
$ git log --pretty=format:"%h %ad | %s%d [%an]" --graph --date=short
* e138e16 2025-02-20 | Second change (HEAD -> master) [Skillet]
* a2b4a11 2025-02-20 | Two changes [Skillet]
* 7427b42 2025-02-20 | v file [Skillet]
* 4b5b90e 2025-02-20 | a+b files [Skillet]
* bba69ea 2025-02-20 | Hello World! [Skillet]

```

Рисунок 17 – Логирование с уточнением формата

Для получение старой версии нашего каталога необходимо узнать для

начала хэши. Они отображены перед датой в примере логирования с форматом. С помощью команды *checkout* проверим первый коммит. Результат выполнения команды представлен на рис. 18.

```
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трнп/repository (master)
$ git checkout bba69ea
Note: switching to 'bba69ea'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false
HEAD is now at bba69ea Hello World!

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трнп/repository ((bba69ea...))
$ |
```

Рисунок 18 – Переключение коммита

Проверка содержимого файла в текущем коммите с помощью команды *cat* представлена на рис. 19.

```
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трнп/repository ((bba69ea...))
$ cat File.html
Hello world!
```

Рисунок 19 – Проверка содержимого файла в текущем коммите

Теперь вернемся в последнее состояние и проверим содержимое файла. Результат представлен на рис. 20.

```

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository ((bba69ea...))
$ git checkout master
Previous HEAD position was bba69ea Hello World!
Switched to branch 'master'

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ cat File.html
<html>
<head>
</head>
  <body>
    <h1>Hello, World!</h1>
  </body>
</html>

```

Рисунок 20 – Возвращение в последнее состояние

Пример отмены изменений до индексации представлен на рис. 21.

```

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ nano File.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   File.html

no changes added to commit (use "git add" and/or "git commit -a")

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ git checkout File.html
Updated 1 path from the index

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ git status
On branch master
nothing to commit, working tree clean

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ cat File.html
<html>
<head>
</head>
  <body>
    <h1>Hello, World!</h1>
  </body>
</html>

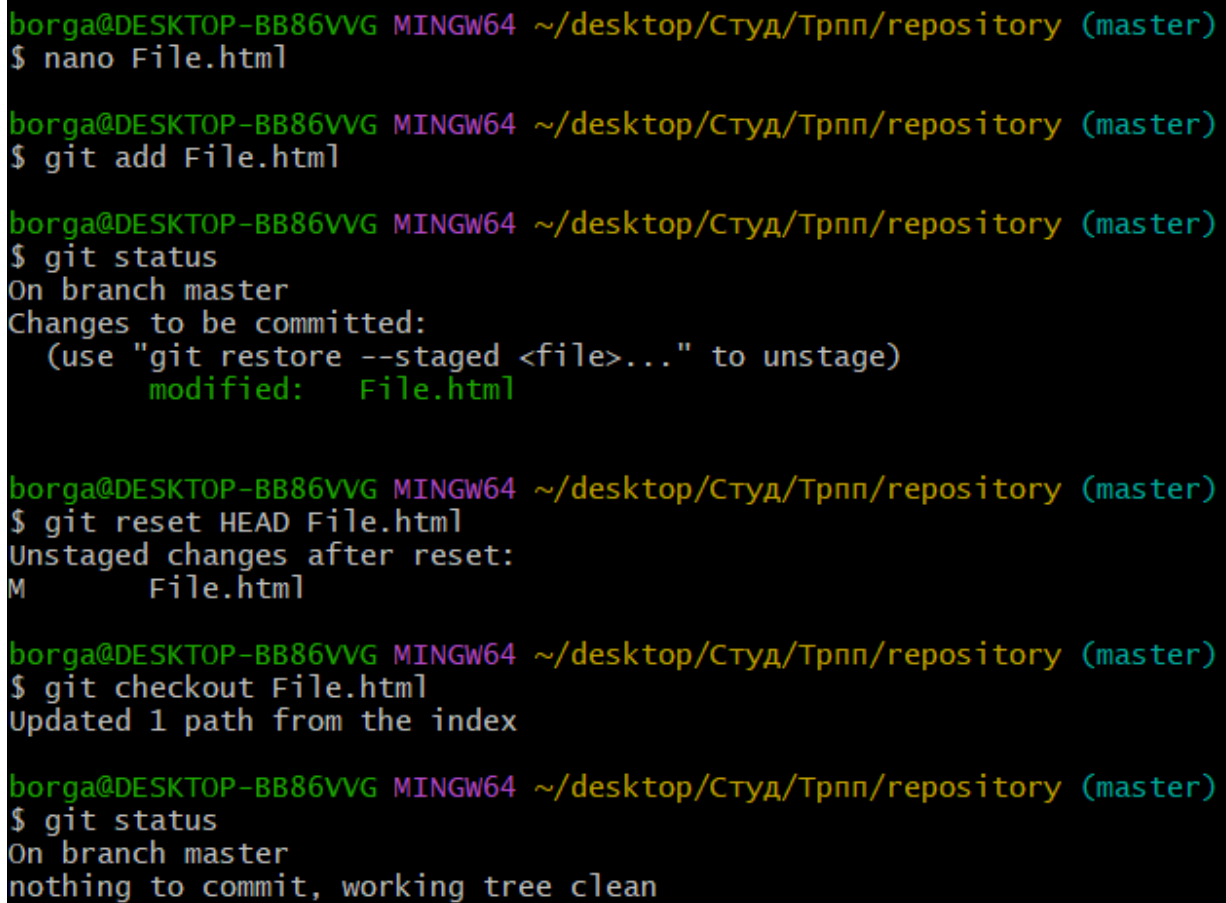
```

Рисунок 21 – Отмена локальных изменений до индексации

Отмена коммитов после индексации производится с помощью команды:

```
git reset HEAD <имя файла>
```

Пример использования команды представлен на рис. 22.



```
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ nano File.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ git add File.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   File.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ git reset HEAD File.html
Unstaged changes after reset:
M       File.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ git checkout File.html
Updated 1 path from the index

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Рисунок 22 – Отмена локальных изменений после индексации

Чтобы отменить коммит, нам необходимо сделать коммит, который удаляет изменения, сохраненные нежелательным коммитом, для это воспользуемся следующей командой:

```
git revert HEAD
```

После выполнения данной команды откроется редактор, который предложит вам отредактировать коммит или вы можете оставить его как есть. Можно также воспользоваться параметром `--no-edit`. Пример удаления коммита представлен на рис. 23.

```

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ nano File.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ git add File.html

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ git commit -m "Лишний коммит"
[master d43ac29] Лишний коммит
1 file changed, 1 insertion(+), 1 deletion(-)

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ git revert HEAD --no-edit
[master 44e2619] Revert "Лишний коммит"
Date: Thu Feb 20 20:30:11 2025 +0300
1 file changed, 1 insertion(+), 1 deletion(-)

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/Студ/Трпп/repository (master)
$ git log --pretty=format:"%h %ad | %s%d [%an]" --graph --date=short
* 44e2619 2025-02-20 | Revert "Лишний коммит" (HEAD -> master) [Skillet]
* d43ac29 2025-02-20 | Лишний коммит [Skillet]
* e138e16 2025-02-20 | Second change [Skillet]
* a2b4a11 2025-02-20 | Two changes [Skillet]
* 7427b42 2025-02-20 | v file [Skillet]
* 4b5b90e 2025-02-20 | a+b files [Skillet]
* bba69ea 2025-02-20 | Hello world! [Skillet]

```

Рисунок 23 – Удаление коммита

Таким образом, в ходе выполнения первой части практической работы были изучены основные команды Git.

Часть 2. Системы управления репозиториями

Для работы с GitHub необходимо зарегистрировать аккаунт. Созданный аккаунт на платформе GitHub представлен на рис. 24.

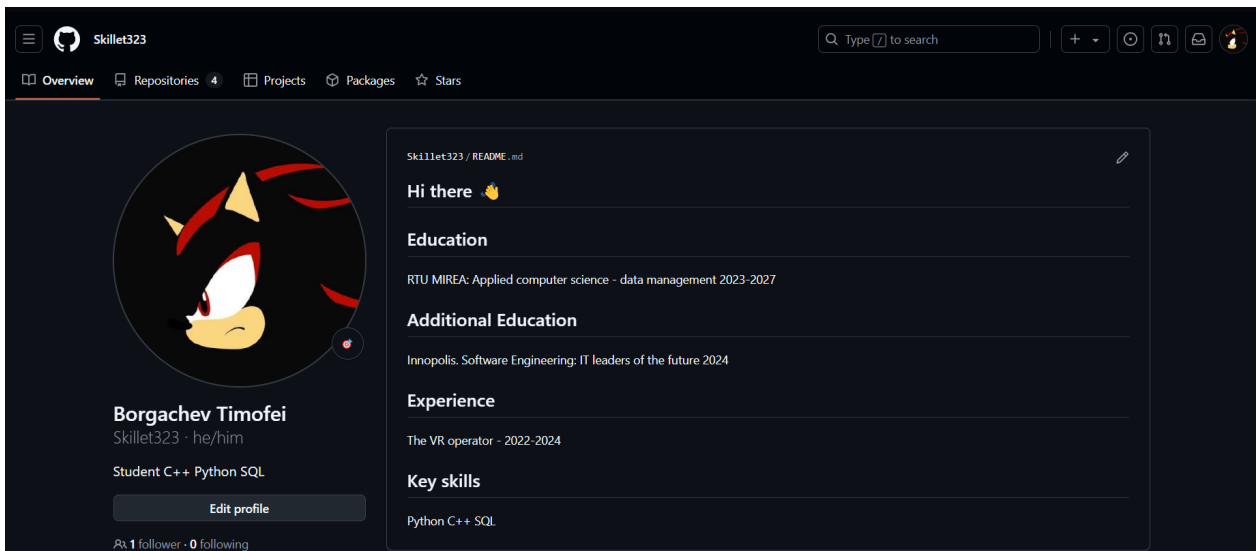


Рисунок 24 – Аккаунт GitHub

Процесс генерации ssh-ключей представлен на рис. 25.

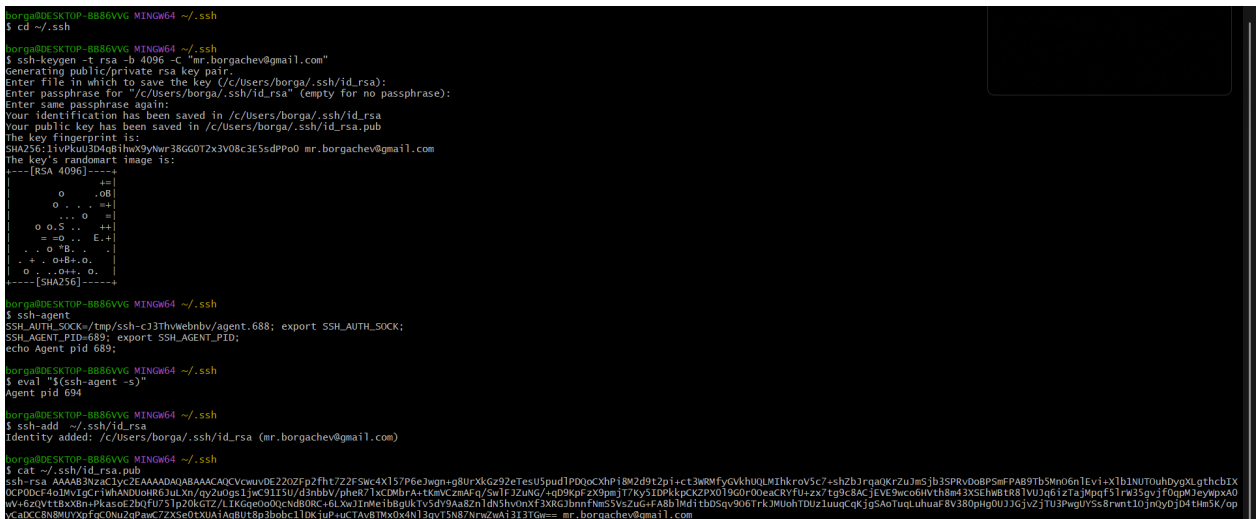


Рисунок 25 – Создание SSH-ключей

Добавленный ключ представлен на рис. 26.

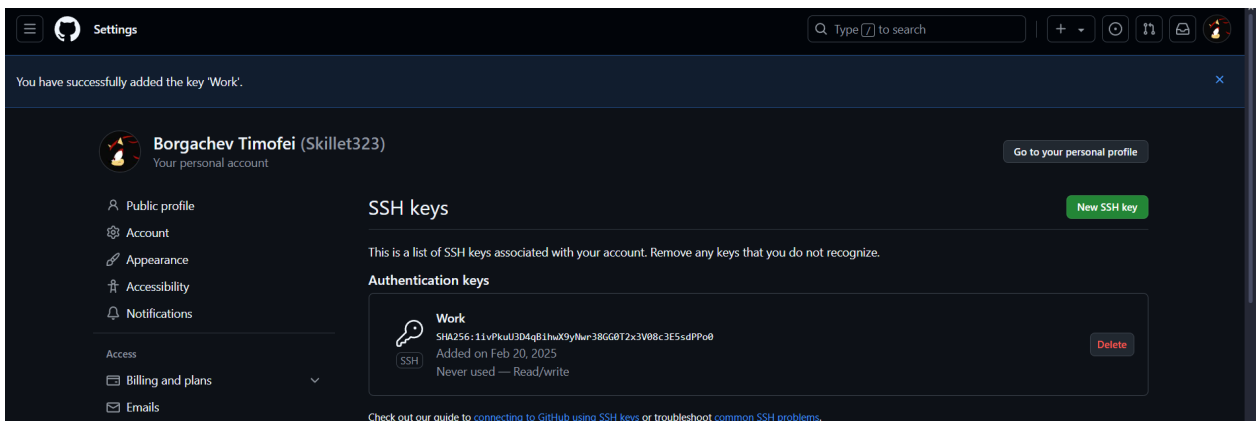


Рисунок 26 – Добавленный SSH-ключ

Для связывания локального репозитория с удаленным – инициализируем новый локальный репозиторий. Результат представлен на рис. 27.

```
borga@DESKTOP-BB86VVG MINGW64 ~  
$ cd ~/Desktop/Студ/ТРПП  
  
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП  
$ mkdir New  
mkdir: cannot create directory 'New': File exists  
  
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП  
$ cd ~/Desktop/Студ/ТРПП  
  
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП  
$ mkdir New_one  
  
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП  
$ cd New_one  
  
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/New_one  
$ touch Short.html  
  
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/New_one  
$ git init  
Initialized empty Git repository in C:/Users/borga/Desktop/Студ/ТРПП/New_one/.git/  
  
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/New_one (master)  
$
```

Рисунок 27 – Инициализация нового локального репозитория

Чтобы связать локальный репозиторий и репозиторий, созданный на GitHub необходимо ввести в консоль следующую команду:

```
git remote add project git@github.com:<ваши имя и  
название репозитория>.git
```

Пример использования команды представлен на рис. 28.

```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/New_one (master)  
$ git remote add project git@github.com:Skillet323/SADT.git
```

Рисунок 28 – Связывание репозитория

Создание новой ветки происходит с помощью одной из команд:

```
git branch <имя ветки>  
git checkout -b <имя ветки>
```

Результат создания новой ветки представлен на рис. 29.

```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/New_one (master)  
$ git branch main
```

Рисунок 29 – Создание новой ветки

Переключение на новую ветку представлено на рис. 30.

```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/New_one (master)
$ git checkout main
Switched to branch 'main'

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/New_one (main)
$ git status
On branch main
nothing to commit, working tree clean
```

Рисунок 30 – Переключение на новую ветку

Пример слияния веток представлен на рис. 31.

```
borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/SADT (main)
$ git checkout Another_one
M       Short.html
Switched to branch 'Another_one'

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/SADT (Another_one)
$ git commit -m "New_short"
[Another_one 8d11022] New_short
1 file changed, 1 insertion(+)

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/SADT (Another_one)
$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

borga@DESKTOP-BB86VVG MINGW64 ~/desktop/студ/трпп/SADT (main)
$ git merge Another_one
Updating bb106af..8d11022
Fast-forward
 Short.html | 1 +
1 file changed, 1 insertion(+)
```

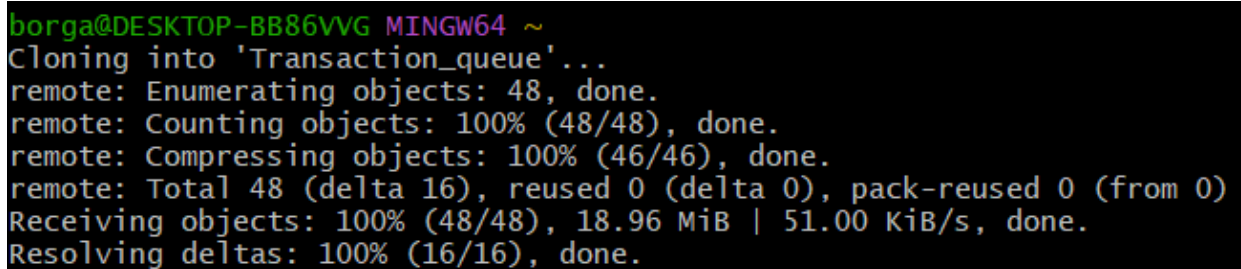
Рисунок 31 – Слияние веток

Согласно индивидуальному варианту, номер 3 необходимо провести следующую цепочку действий:

- Клонировать непустой удаленный репозиторий на локальную машину
- Создать тег, указывающий на последний коммит в ветке master
- Создать новую ветку и вывести список всех веток
- Произвести 3 коммита в новой ветке

- Выгрузить все изменения в удаленный репозиторий
- Откатить ветку к созданному тегу (в том числе в удаленном репозитории)
- Вывести в консоли различия между веткой master и новой веткой

Клонирование удаленного репозитория с помощью команды *git clone* *<ссылка на репозиторий>* представлено на рис. 32.



```
borga@DESKTOP-BB86VVG MINGW64 ~  
Cloning into 'Transaction_queue'...  
remote: Enumerating objects: 48, done.  
remote: Counting objects: 100% (48/48), done.  
remote: Compressing objects: 100% (46/46), done.  
remote: Total 48 (delta 16), reused 0 (delta 0), pack-reused 0 (from 0)  
Receiving objects: 100% (48/48), 18.96 MiB | 51.00 KiB/s, done.  
Resolving deltas: 100% (16/16), done.
```

Рисунок 32 – Клонирование репозитория

Создание тега для последнего коммита представлено на рис. 33.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/Variant/Transaction_queue (main)
$ git log --pretty=format:"%h | %s"
bccee19 | Update Instruction.md
f372367 | Update Instruction.md
f561f01 | Update Instruction.md
7c46a51 | Update Instruction.md
150b9e9 | Update Instruction.md
f8caf80 | Docker_container
d5507c0 | Create Instruction.md
d22a21d | Update README.md
c0907d0 | Update README.md
77e448e | Update README.md
acaf498 | Example of result
de129af | Updated version
76c4887 | Main_file
79e460b | Create README.md

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/Variant/Transaction_queue (main)
$ git checkout bccee19
Note: switching to 'bccee19'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at bccee19 Update Instruction.md

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/Variant/Transaction_queue ((bccee19...))
$ git tag v1

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/Variant/Transaction_queue ((v1))

```

Рисунок 33 – Создание тега последнего коммита

Добавление новой ветки и вывод всех веток представлены на рис. 34.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/Variant/Transaction_queue (master)
$ git checkout -b test
Switched to a new branch 'test'

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/Variant/Transaction_queue (test)
$ git branch --all
  main
  master
* test
remotes/origin/Features
remotes/origin/HEAD -> origin/main
remotes/origin/ML_Model
remotes/origin/main

```

Рисунок 34 – Вывод всех веток репозитория

Произведение трех коммитов в новой ветке представлено на рис. 35.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Variant/Transaction_queue (test)
$ nano Main.ipynb

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Variant/Transaction_queue (test)
$ git add Main.ipynb

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Variant/Transaction_queue (test)
$ git commit -m "Some updates"
[test 6520d7d] Some updates
1 file changed, 1 insertion(+), 1 deletion(-)

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Variant/Transaction_queue (test)
$ nano README.md

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Variant/Transaction_queue (test)
$ git add README.md

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Variant/Transaction_queue (test)
$ git commit -m "README update"
[test 769ed91] README update
1 file changed, 1 insertion(+), 1 deletion(-)

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Variant/Transaction_queue (test)
$ nano results.csv

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Variant/Transaction_queue (test)
$ git add results.csv
gf
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Variant/Transaction_queue (test)
$ git commit -m "New update"
[test e2caa8b] New update
1 file changed, 1 insertion(+), 1 deletion(-)

```

Рисунок 35 – Три коммита в новую ветку

Выгрузка изменений в удаленный репозиторий представлена на рис. 36.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Variant/Transaction_queue (test)
$ git push origin test
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 12 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 1.22 KiB | 311.00 KiB/s, done.
Total 10 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 4 local objects.
remote:
remote: Create a pull request for 'test' on GitHub by visiting:
remote:   https://github.com/Skillet323/Transaction_queue/pull/new/test
remote:
To github.com:Skillet323/Transaction_queue.git
* [new branch]      test -> test

```

Рисунок 36 – Выгрузка изменений в удаленный репозиторий

Откат ветки к созданному тегу представлен на рис. 37.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/Variant/Transaction_queue (test)
$ git reset --hard v1
HEAD is now at bccee19 Update Instruction.md

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/Variant/Transaction_queue (test)
$ git push origin test --force
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Skillet323/Transaction_queue.git
+ e2caa8b...bccee19 test -> test (forced update)

```

Рисунок 37 – Откат ветки к созданному тегу в локальном и удаленном репозиториях

Вывод различий между новой веткой и веткой мастер представлен на рис. 38.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/Variant/Transaction_queue (test)
$ git diff master test
diff --git a/Main.ipynb b/Main.ipynb
index 0a49dd5..be83aec 100644
--- a/Main.ipynb
+++ b/Main.ipynb
@@ -152,7 +152,7 @@
     "
     "         features.append((feature, success))\n",
     "         return features\n",
     "     "\n",
-    "# Updated functions to predict flow and create chains\n",
+    "# Updated function to predict flow and create chains\n",
+    "def predict_flow(model, transaction, providers, rates, features_df) -> pd.DataFrame:\n",
     "     try:\n",
     "         results = []\n",

```

Рисунок 38 – Различия в ветках

Таким образом, в ходе выполнения второй части работы, были получены навыки работы с удаленным репозиторием.

Часть 3. Работа с ветвлением и оформление кода

Пример форка репозитория согласно варианту 3 представлен на рис. 39.

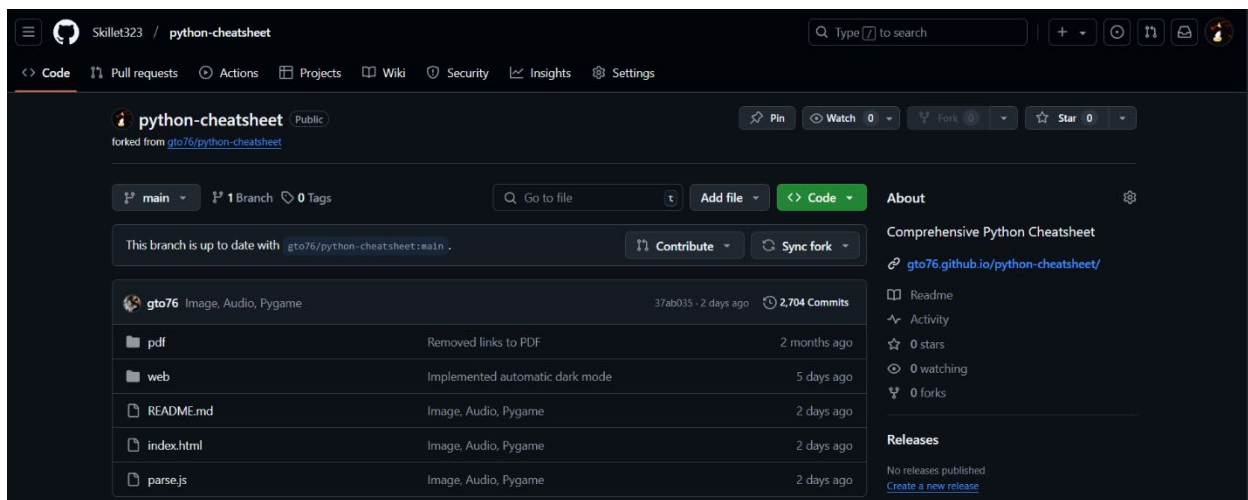


Рисунок 39 – Форк репозитория

Клонирование репозитория в локальный представлено на рис. 40.

```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ git clone git@github.com:Skillet323/python-cheatsheet.git
Cloning into 'python-cheatsheet'...
remote: Enumerating objects: 10220, done.
remote: Counting objects: 100% (113/113), done.
remote: Compressing objects: 100% (59/59), done.
remote: Total 10220 (delta 62), reused 58 (delta 54), pack-reused 10107 (from 2)
Receiving objects: 100% (10220/10220), 12.18 MiB | 148.00 KiB/s, done.
Resolving deltas: 100% (6073/6073), done.
```

Рисунок 40 – Клонирование удаленного репозитория

Процесс создания двух новых веток в репозитории представлен на рис. 41.

```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ cd python-cheatsheet

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (main)
$ git branch branch1

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (main)
$ git branch branch2

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (main)
$ git checkout branch1
Switched to branch 'branch1'

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ |
```

Рисунок 41 – Создание двух новых веток

Добавление трех коммитов в первую ветку представлен на рис. 42.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ touch File1.txt

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ git add File1.txt

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ git commit -m "Add File1"
[branch1 c560b46] Add File1
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 File1.txt

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ nano index.html

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ git add index.html

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ git commit -m "Change index"
[branch1 06f7789] Change index
1 file changed, 1 insertion(+)

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ nano parse.js

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ git add parse.js

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ git commit -m "Change parse.js"
[branch1 0296009] Change parse.js
1 file changed, 1 insertion(+), 1 deletion(-)

```

v

Рисунок 42 – Добавление трех коммитов в ветку branch1

Процесс добавления трех коммитов в ветку branch2 представлен на рис.

43.


```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch1)
$ git checkout branch2
Switched to branch 'branch2'

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch2)
$ touch File2.txt

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch2)
$ git add File2.txt
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch2)
$ git commit -m "Add File2"
[branch2 49a5ca1] Add File2
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 File2.txt

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch2)
$ touch File3.txt

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch2)
$ git add File3.txt

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch2)
$ git commit -m "Add File3"
[branch2 9d8e361] Add File3
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 File3.txt

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch2)
$ nano parse.js

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch2)
$ git add parse.js

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch2)
$ git commit -m "Change parse"
[branch2 ce977ed] Change parse
1 file changed, 1 insertion(+), 1 deletion(-)

```

Рисунок 43 – Добавление трех коммитов в ветку branch2

Пример слияния двух веток представлен на рис. 44.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch2)
$ git checkout branch1
Switched to branch 'branch1'

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch1)
$ git merge branch2
Auto-merging parse.js
Merge made by the 'ort' strategy.
 File2.txt | 0
 File3.txt | 0
 parse.js  | 2 +-
3 files changed, 1 insertion(+), 1 deletion(-)
create mode 100644 File2.txt
create mode 100644 File3.txt

```

Рисунок 44 – Слияние двух веток

Процесс выгрузки всех изменений из всех веток представлен на рис. 45.

```

barga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch1)
$ git push origin branch1
Enumerating objects: 22, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 12 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (19/19), 1.79 KiB | 366.00 KiB/s, done.
Total 19 (delta 11), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (11/11), completed with 2 local objects.
remote:
remote: Create a pull request for 'branch1' on GitHub by visiting:
remote:   https://github.com/Skillet323/python-cheatsheet/pull/new/branch1
remote:
To github.com:Skillet323/python-cheatsheet.git
 * [new branch]      branch1 -> branch1

barga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch1)
$ git checkout branch2
Switched to branch 'branch2'
g
barga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch2)
$ git push origin branch2
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'branch2' on GitHub by visiting:
remote:   https://github.com/Skillet323/python-cheatsheet/pull/new/branch2
remote:
To github.com:Skillet323/python-cheatsheet.git
 * [new branch]      branch2 -> branch2

barga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch2)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

barga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (main)
$ git push origin main
Everything up-to-date

```

Рисунок 45 – Выгрузка изменений всех веток в удаленный репозиторий
 Добавление еще трех коммитов в branch1 представлено на рис. 46.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (main)
$ git checkout branch1
Switched to branch 'branch1'

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ touch test

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ git add test

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ git commit -m "Add test"
[branch1 3920069] Add test
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 test

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ nano parse.js

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ git add parse.js

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ git commit -m "Another change parse"
[branch1 ea49fb7] Another change parse
1 file changed, 1 insertion(+), 1 deletion(-)

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ nano README.md

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ git add README.md

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ git commit -m "Change README"
[branch1 e2ed891] Change README
1 file changed, 1 insertion(+), 1 deletion(-)

```

Рисунок 46 – Новые коммиты в branch1

Процесс повторного клонирования репозитория представлен на рис. 47.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/python-cheatsheet (branch1)
$ cd ~/Desktop/Студ/ТРПП/

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ mkdir Test

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП
$ cd Test

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Test
$ git clone git@github.com:Skillet323/python-cheatsheet.git
Cloning into 'python-cheatsheet'...
remote: Enumerating objects: 10239, done.
remote: Counting objects: 100% (131/131), done.
remote: Compressing objects: 100% (68/68), done.
remote: Total 10239 (delta 73), reused 75 (delta 63), pack-reused 10108 (from 2)
Receiving objects: 100% (10239/10239), 12.19 MiB | 224.00 KiB/s, done.
Resolving deltas: 100% (6084/6084), done.

```

Рисунок 47 – Повторное копирование репозитория в другую директорию

Дополнительные 3 коммита в ветку branch1 в новой директории представлены на рис. 48.

```
borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Test
$ cd python-cheatsheet

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Test/python-cheatsheet (main)
$ git checkout -b branch1
Switched to a new branch 'branch1'

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Test/python-cheatsheet (branch1)
$ touch File4.pdf

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Test/python-cheatsheet (branch1)
$ git add File4.pdf

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Test/python-cheatsheet (branch1)
$ git commit -m "Add File4"
[branch1 9bb6d98] Add File4
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 File4.pdf

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Test/python-cheatsheet (branch1)
$ touch File5.docx

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Test/python-cheatsheet (branch1)
$ git add File5.docx

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Test/python-cheatsheet (branch1)
$ git commit -m "Add File5"
[branch1 e598bfb] Add File5
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 File5.docx

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Test/python-cheatsheet (branch1)
$ nano index.html

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Test/python-cheatsheet (branch1)
$ git add index.html

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТРПП/Test/python-cheatsheet (branch1)
$ git commit -m "Change index.html"
[branch1 5cce303] Change index.html
1 file changed, 1 insertion(+), 1 deletion(-)
```

Рисунок 48 – Добавление обособленных трех коммитов в ветку branch1

Выгрузка из новой директории представлена на рис. 49.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/Test/python-cheatsheet (branch1)
$ git push origin branch1
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 12 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 1.05 KiB | 535.00 KiB/s, done.
Total 10 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 3 local objects.
To github.com:Skillet323/python-cheatsheet.git
   369a3c5..4ddb829  branch1 -> branch1

```

Рисунок 49 – Выгрузка из новой директории

Процесс принудительной выгрузки из старого репозитория представлен на рис. 50.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch1)
$ git push origin branch1 --force
Enumerating objects: 27, done.
Counting objects: 100% (25/25), done.
Delta compression using up to 12 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (19/19), 1.84 KiB | 314.00 KiB/s, done.
Total 19 (delta 11), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (11/11), completed with 3 local objects.
To github.com:Skillet323/python-cheatsheet.git
 + 4ddb829...e2ed891 branch1 -> branch1 (forced update)

```

Рисунок 50 – Принудительная выгрузка из старого репозитория

Полученные командой *git pull* изменения представлены на рис. 51.

```

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/python-cheatsheet (branch1)
$ cd ~/Desktop/Студ/ТПП/Test/python-cheatsheet

borga@DESKTOP-BB86VVG MINGW64 ~/Desktop/Студ/ТПП/Test/python-cheatsheet (branch1)
$ git pull origin branch1
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 8 (delta 5), reused 7 (delta 4), pack-reused 0 (from 0)
Unpacking objects: 100% (8/8), 778 bytes | 9.00 KiB/s, done.
From github.com:Skillet323/python-cheatsheet
 * branch          branch1      -> FETCH_HEAD
 + 4ddb829...e2ed891 branch1     -> origin/branch1  (forced update)
Merge made by the 'ort' strategy.
 README.md | 2 +-
 parse.js  | 2 +-
 test      | 0
3 files changed, 2 insertions(+), 2 deletions(-)
create mode 100644 test

```

Рисунок 51 – Использование команды *git pull*

Таким образом, в ходе выполнения третьей части были получены навыки по работе с ветвлением.

Ответы на вопросы

1. Что такое репозиторий Git?
 - Репозиторий Git — это место, где хранятся все файлы проекта вместе с их историей изменений. Он может быть локальным или удаленным.
2. Что такое тег в репозитории Git?
 - Тег — это метка, указывающая на конкретный коммит, обычно используется для обозначения релизов.
3. Как отменить слияние веток, если произошел конфликт?
 - С помощью команды:
`git merge --abort`
4. Что делает команда *git add*?
 - Индексирует файлы чтобы позже сделать коммит
5. Чем отличаются команды *git push* и *git pull*?
 - *git push* отправляет локальные изменения в удаленный репозиторий, а *git pull* забирает изменения из удаленного репозитория.
6. Чем отличается *master* и *origin/master*?
 - *master* — локальная ветка, *origin/master* — отражение удаленной ветки на удаленном репозитории.
7. Как узнать, кто автор строки в файле, используя систему Git?
 - с помощью команды *git blame <файл>*.
8. Как удалить все untracked файлы?
 - с помощью команды *git clean -fd*.

Вывод

В ходе выполнения практической работы были успешно изучены и применены основные команды Git для работы с локальными и удаленными

репозиториями. Были получены навыки по созданию, индексации и коммиту изменений, управлению ветками, а также откату к предыдущим версиям проекта. Работа с GitHub позволила закрепить знания о синхронизации локальных и удаленных репозиториях. Полученные навыки являются важной частью профессиональной деятельности при разработке программных приложений и работе в команде.